Advanced Agentic AI Systems Engineering

Capstone Rubric & Submission Requirements

SDAIA Academy, delivered via Learning Space | 5-day advanced capstone, on-site | 30 training hours

1. Capstone Rubric

100 points total | Pass mark: 60 or above. No single deliverable may score below 40% of its points - a strong project cannot compensate for a deliverable that was skipped entirely. The capstone integrates all five days into one deployable agentic system.

#	Deliverable	Pts	What is required
1	**Agentic Reasoning & Tool Use**	**15**	A working agent that reasons and calls real tools/functions (not hardcoded outputs) via function calling or an MCP-style tool interface. Must name and implement at least one explicit reasoning pattern from the course: ReAct (Thought->Action->Observation), Plan-and-Execute, Reflexion/self-critique, or Hierarchical Delegation. Short-term memory (state carried across steps) is in place.
2	**Graph-Based Orchestration**	**20**	A genuine state-graph workflow (LangGraph StateGraph, AutoGen, or equivalent) with nodes, edges, and at least one conditional/branching edge - not a linear hardcoded chain relabeled as an agent. State is a real shared object read and updated by nodes, and the graph supports a loop (e.g. retry, re-plan, or re-search) that terminates on a condition.
3	**Multi-Agent System & Role Specialization**	**20**	Two or more agents with distinct, named responsibilities (e.g. Planner, Researcher, Coder, Reviewer, Coordinator) that communicate through structured messages or shared state - not one agent role-playing multiple personas in a single prompt. A coordination strategy is explicit: centralized/coordinator, hierarchical delegation, or decentralized handoff.
4	**Security, Guardrails & Observability**	**20**	At least one demonstrated input guardrail (e.g. prompt-injection detection/blocking) with a real attack attempt shown being caught, plus an output or data-protection guardrail (PII masking, output validation, or policy filter). Paired with structured monitoring: logs, metrics, or tracing (LangSmith, Arize Phoenix, Prometheus, or equivalent) that capture tool calls, latency, cost, or failures - not print statements.
5	**Production Readiness: Persistence, HITL & Cloud**	**20**	A persistent checkpointer (SqliteSaver, PostgresSaver, Redis, or equivalent) that survives a restart, so long-running state is not lost. At least one real human-in-the-loop interrupt/approval node that pauses the graph and resumes on human input. A cloud deployment story with an artifact to prove it (Dockerfile, docker-compose, IaC snippet, or a simulated cloud service such as MinIO/S3, Redis, or a FastAPI endpoint) - not just a paragraph claiming it is "cloud-ready."
6	**Documentation & Evidence of Execution**	**5**	Executed notebook or logs with real captured output for every deliverable above - not just code that could theoretically run. A short architecture write-up using the course's own vocabulary (nodes, edges, state, agents, tools) that explains what was built and why.

How it is evaluated

- **Use a real orchestration framework. Credit is given for LangGraph, AutoGen, CrewAI, Semantic Kernel, Haystack, or a comparably real graph/multi-agent library. A hand-rolled if/else chain calling itself an "agent workflow" does not satisfy Deliverable 2.**
- **A simulation does not earn the deliverable, however well written. A single-prompt script pretending to be multi-agent, a "guardrail" that is only a code comment with no enforcement, or a claimed cloud deployment with no artifact will not be credited.**
- **Prove the failure and security paths, not just the happy path. Show an actual blocked prompt-injection attempt, an actual retry or fallback firing on a simulated failure, and an actual human-approval pause that resumes correctly.**
- **Run your code and keep the output. Executed notebooks with captured output, or logs of a real run, are the evidence that each stage actually works - not just that the code exists.**

2. GitHub & Documentation Requirements

These apply to every project, in addition to the rubric above. They are part of how projects are evaluated.

2.1 Mandatory

- Every trainee must **create and activate a GitHub account** if they do not already have one.
- All AI-related training projects must be **uploaded to GitHub**, kept documented and continuously updated.
- A project not published to GitHub as described here is **not a complete submission**.

2.2 Every project repository must include

Requirement	What it means in practice
Clear, comprehensive project description	What the agentic system does, the problem it solves, its architecture (single-agent or multi-agent), and its scope - visible from the repo landing page.
Professional README	Explains the project idea and how to run and use it: prerequisites, API keys/environment variables needed, install/setup steps, how to execute, and expected output.
Proper technical documentation	An architecture or agent-graph overview (nodes/edges/agents/tools), key components/modules, and any configuration required.
Good Git version-control practices	Meaningful commit messages, incremental commit history (not a single bulk upload), sensible repo structure, and a .gitignore that excludes secrets, API keys, and generated files.
Training program attribution	State clearly which training program the project was completed under (program name, and the cohort/session dates).
Link to SDAIA Academy on GitHub	Reference https://github.com/SDAIAAcademy in the README where relevant.

2.3 Encouraged: supporting the Saudi tech community

Trainees are encouraged to support outstanding Saudi projects on GitHub by:

- **Starring** high-quality repositories.
- **Following** Saudi accounts and repositories.
- **Contributing** to open-source projects.
- **Engaging** through Fork, Pull Requests, and Issues where appropriate.
- **Sharing** standout projects with the wider tech community.

Note: community engagement is encouraged and looked on favourably, but it is not scored against the 100-point rubric.